



STOCKBACK: Signed Purchase Evidence and Admin-less Ownership Settlement on Arbitrum Stylus

Every receipt, sealed. Scan. Prove. Own.

Project STOCKBACK · Arbitrum Open House Singapore Buildathon 2026

Live: stockbacks.vercel.app · Robinhood Chain testnet (chain 46630)

Source: github.com/notwen123/STOCKBACK

Abstract. Reward programs that pay on a purchase must first believe the purchase happened, and almost all of them believe a receipt photograph. That belief no longer holds: in a 2026 paired study, people picked an AI-edited receipt from its authentic counterpart at chance (0.501), and the best forensic detectors reached an AUC of 0.53–0.60 [1]. We present *STOCKBACK*, a protocol that moves the trust origin of a purchase from pixels to a signature and settles each verified purchase as shares of a brand-specific, admin-less ERC-4626 vault. A merchant point-of-sale signs every reward-relevant field of a receipt with Ed25519; an attester accepts the receipt only if that signature verifies, the receipt has not expired, the brand is supported and the receipt has never been claimed, then signs a hashes-only EIP-712 claim bound to the claimant’s wallet. On-chain, a single transaction checks the attestation in Rust on Arbitrum Stylus, burns a per-receipt nullifier, applies eligibility, reward caps and a sponsor budget, and deposits into the vault on the user’s behalf. We formalize the claim calculus, prove replay resistance and a closed-form bound on the loss from an attester-key compromise, and report measurements on Robinhood Chain testnet. Verifying Ed25519 in Stylus costs 65,088 gas per signature against 608,542 for the best available Solidity implementation (both linear fits, $R^2 > 0.9999$), up to 9.2× cheaper, and fits 100 signatures in one transaction where Solidity cannot fit 53. The reference deployment has settled 12 real claims; it is covered by 83 Foundry tests including fuzzing and invariants, 5 Rust tests, 42 receipt-signing and live-integration tests and an 11-step browser suite. We state plainly what is simulated (the merchant), what is mocked (every brand asset) and what is not yet built.

Keywords: proof of purchase; receipt forgery; Ed25519; Arbitrum Stylus; EIP-712; nullifiers; ERC-4626; loyalty programs; Robinhood Chain.

1 Introduction

A purchase leaves evidence behind: a receipt, a UPI reference, an order email. Every system that rewards purchases, refunds damaged goods or reimburses expenses converts that evidence into money, and nearly all of them accept the evidence in its weakest form, an image. Generative image models have changed the cost of that choice. Changing one number on a receipt photograph now takes “under a second for a few cents”; when shown an authentic receipt and its AI-edited twin side by side, 120 human inspectors chose correctly 50.1% of the time; forensic detectors that score 0.85–0.96 AUC on traditional tampering fall to 0.585–0.599 on the same documents edited by a modern model [1]. Refund-fraud evidence shows the same pattern: multimodal models catch only 23% of AI-fabricated damage photos while specialized detectors falsely accuse honest customers [3]. The economics are well understood: when fabrication becomes cheap before verification becomes worthwhile, high-trust markets that delay checking attract fraud first [4]. A new reward program that trusts photographs is exactly such a market.

The reward side is also weak. Consumers belong to 14.8 loyalty programs on average and use 6.7; 73% find them too complicated; roughly 60% of coalition programs fail within ten years, and operators can expire or devalue balances at will [7]. Points are a liability on someone else’s balance sheet, not property.

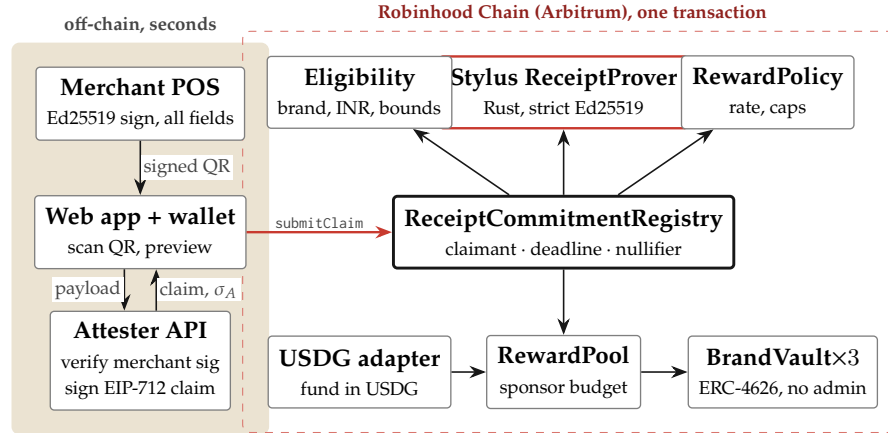


Figure 1: STOCKBACK architecture. The registry is the only contract that consumes caps or spends budget; the verifier only answers yes or no; vaults have no administrator. Everything inside the dashed box runs in the single `submitClaim` transaction.

STOCKBACK addresses both halves. On the evidence side, trust moves from the image to a signature produced at the point of sale, and every claim carries an explicit *evidence tier* that states what was and was not established. On the settlement side, a verified purchase becomes shares of an ERC-4626 vault with no owner, no pause and no sweep: once minted, only the holder can move them.

1.1 Contributions

- A **claim calculus** (Section 4): an EIP-712 purchase claim whose identifiers are hashed off-chain, a nullifier independent of claimant, amount and deadline, and an acceptance predicate evaluated in one transaction, with a proof of replay resistance and a closed-form bound on attester-compromise loss.
- **Merchant-signed receipts** (Section 5): a versioned canonical message over eight fields, field grammars that make line injection impossible, and an attester that verifies the merchant signature against a public key it holds, never the private key.
- **Ed25519 on Arbitrum Stylus** (Section 6): a Rust verifier behind the same ABI as the Solidity verifier, with a measured linear gas model and the resulting batch ceilings under the 32M per-transaction cap.
- An **evaluation against the live chain** (Section 8) and **findings** that only building and testing against deployed code surfaced (Section 9), including one defect in the upstream code this project forked.

2 Threat Model and Design Goals

We consider six principals. The *shopper* holds a wallet and is untrusted. The *merchant point-of-sale (POS)* holds an Ed25519 signing key and is trusted to sign only sales that happened; in this deployment it is simulated. The *attester* holds the merchant public key and its own Ed25519 key, and is trusted for the checks it performs, not beyond them. The *registry and verifier* are contracts, trusted for integrity of their code. The *sponsor* funds a per-brand budget and is trusted only with its own funds. The *owner* configures policies and caps and is the single administrative key (Section 10).

Goals. (G1) **Field binding:** no reward-relevant field (brand, amount, currency, receipt, merchant, time) can change between the point of sale and the chain without invalidating a signature. (G2) **Once per receipt:** a receipt yields at most one reward, for anyone, regardless of claimant, amount or deadline. (G3) **Bounded authority:** every key, including the attester’s and the owner’s, can lose at most a capped, pre-funded amount; no key can touch shares already minted. (G4) **No personal data on-chain:** only hashes, an amount, a currency and timestamps. (G5) **Honest evidence:** every claim states its evidence tier; a photograph is never presented as proof of purchase. (G6) **Fail closed:** a missing key or an unverifiable receipt stops the claim;

nothing is signed on a guess.

Non-goals. Proof of personhood (one human, one wallet); protection against a compromised merchant key; recovering rewards for refunded purchases (no hold period yet); and confidentiality of the purchase amount.

3 System Architecture

Figure 1 shows the deployed system: eight Solidity contracts (OpenZeppelin 5, solc 0.8.24, 791 lines), one Rust contract on Stylus (251 lines), and a Next.js web application with the attester and simulated POS as server routes. Contracts are single-purpose and acyclic; none is upgradeable. Table 1 gives the authority analysis: which component can produce which effect.

Table 1: Authority analysis. Each row is a component, each column an effect. The last column is empty by construction: no code path lets any key move shares a user already holds.

Component	Sign claim	Burn nullifier	Spend budget	Change rules	Move user shares
Merchant POS key	receipt only	—	—	—	—
Attester key	yes	—	—	—	—
Stylus verifier	—	—	—	—	—
Registry (code)	—	yes	via pool	—	—
RewardPool (code)	—	—	yes ^a	—	—
Owner key	—	—	unallocated only	yes	—
Shopper wallet	—	—	—	—	own only

^aOnly when called by the registry; `allocate` reverts for any other caller.

3.1 Evidence tiers

The on-chain path is identical for every claim. What differs is what the attester checked before signing, and STOCKBACK labels it on every screen (Table 2).

Table 2: Evidence tiers.

Tier	Evidence	What it establishes	Status
1	Merchant-signed receipt (QR)	Every field unaltered since the merchant key signed it. Merchant simulated in this deployment.	Live demo
2	Attested photo / OCR	Only that the attester signed what it was given; authenticity is not established.	Live demo
3	Payment or order proof (zkTLS)	Provenance from the payment or order source itself [8].	Roadmap

4 The Claim Calculus

4.1 Claims and commitments

A purchase claim is the tuple

$$c = (w, b, m, r, a, \kappa, t_p, t_d), \quad (1)$$

where w is the claimant’s address, b the brand identifier, $m = \text{keccak256}(\text{merchant})$, a the amount in minor units (paise), κ an ISO-4217 code, t_p the purchase time and t_d the attestation deadline. The receipt reference ρ never appears; the claim carries a salted hash

$$r = \text{keccak256}(\text{abi.encode}(\text{keccak256}(s), \rho)), \quad (2)$$

where s is a secret attester salt, so short references such as an eight-digit invoice number cannot be brute-forced from public data. The claim's commitment and the digest the attester signs are

$$H(c) = \text{keccak256}(\text{abi.encode}(\tau_c, w, b, m, r, a, \kappa, t_p, t_d)), \quad (3)$$

$$\text{DS} = \text{keccak256}(\text{abi.encode}(\tau_D, \text{"STOCKBACK"}, "1", \chi, \mathcal{R})), \quad (4)$$

$$d(c) = \text{keccak256}(\text{0x1901} \parallel \text{DS} \parallel H(c)), \quad (5)$$

with τ_c, τ_D the EIP-712 type hashes [9], χ the chain identifier and $\mathcal{R}$ the registry address. Binding χ and $\mathcal{R}$ into the domain makes a signature useless on any other chain or deployment.

4.2 Nullifiers

Definition 1 (Nullifier). For a claim c ,

$$v(c) = \text{keccak256}(\text{abi.encode}(\text{keccak256}(\text{"STOCKBACK_NULLIFIER_V1"}), m, r)).$$

The nullifier depends only on the merchant and the salted receipt hash. It deliberately excludes the claimant, amount, brand and deadline, so none of them can be varied to re-use a receipt.

Proposition 1 (Once per receipt). *Let N be the registry's set of burned nullifiers. For any two accepted claims $c \neq c'$ with the same merchant and receipt reference, at most one is accepted. More strongly, $v(c) = v(c')$ for every pair of claims that agree on (m, r) , regardless of $(w, b, a, \kappa, t_p, t_d)$.*

Proof. v is a function of (m, r) alone. Acceptance requires $v(c) \notin N$ and inserts $v(c)$ into N before any external call (checks-effects-interactions), within one transaction under a reentrancy guard. The second claim therefore observes $v(c') = v(c) \in N$ and is rejected with `NULLIFIERUSED`. Collisions of distinct (m, r) pairs require a keccak256 collision. The property is exercised by unit tests, a 1,000-run fuzz test over claimant, amount and deadline, and the invariant `invariant_noReplay`. $\square$

4.3 Acceptance

Let $\text{Verify}_V(d, \sigma)$ be the active verifier (Ed25519 in Stylus or ECDSA in Solidity) with attester allowlist V . A claim with attestation σ submitted by sender u at block time t is accepted iff

$$\text{Accept}(c, \sigma) \iff \underbrace{u = w}_{\text{binding}} \wedge \underbrace{t \leq t_d}_{\text{fresh}} \wedge \underbrace{v(c) \notin N}_{\text{once}} \wedge \underbrace{\text{Verify}_V(d(c), \sigma)}_{\text{attested}} \wedge \underbrace{E_b(c)}_{\text{eligible}} \wedge \underbrace{Q_b(c)}_{\text{within caps}} \wedge \underbrace{R(c) \leq B_b}_{\text{funded}}, \quad (6)$$

evaluated in exactly that order, the first failing conjunct naming one of sixteen status codes. Eligibility is

$$E_b(c) \iff \text{active}_b \wedge \kappa = \kappa_b \wedge a_b^{\min} \leq a \leq a_b^{\max} \wedge t_p \leq t \wedge t - t_p \leq T_{\text{age}}, \quad (7)$$

with live values $a^{\min} = \text{₹}100$, $a^{\max} = \text{₹}5,00,000$ and $T_{\text{age}} = 30$ days. Because the order is fixed and every check before the transfer is a pure function of state, `previewClaim` returns exactly the status that `submitClaim` would revert with, which the interface uses to explain a rejection before the wallet opens.

4.4 Rewards and caps

For brand b with rate ρ_b (asset base units per minor currency unit, 10^{18} -scaled) and multiplier $\mu_b \in (0, 3]$ (in basis points),

$$R(c) = \min\left(\left\lfloor \frac{a \rho_b \mu_b}{10^{18} \cdot 10^4} \right\rfloor, C_b^{\text{claim}}\right). \quad (8)$$

The product is computed with a 512-bit intermediate (`Math.mulDiv`), so it cannot overflow for any $a < 2^{128}$. Daily caps *reject* rather than *clip*:

$$Q_b(c) \iff U_b(w, \delta) + R(c) \leq C_b^{\text{user}} \wedge D_b(\delta) + R(c) \leq C_b^{\text{brand}}, \quad \delta = \lfloor t/86400 \rfloor. \quad (9)$$

Rejecting matters: a clipped claim would burn the nullifier for a partial reward, while a rejected claim leaves the receipt claimable on a later day. Table 3 lists the live parameters.

For example, a ₹2,000 Nike purchase has $a = 200,000$ paise and $\rho = 7.5 \times 10^{13}$, giving $R = 15$ units, the value observed in every Nike claim on chain.

Table 3: Live reward parameters on Robinhood Chain testnet, read from `RewardPolicy` and `RewardPool` on 2 October 2026. Units are demo asset units (10^{18} base units).

Brand	Effective rate	C^{claim}	C^{user} per day	C^{brand} per day	Budget B_b
NIKE	0.75% of value	100	300	100,000	$\approx 1.01 \times 10^6$
AAPL	0.50% of value	100	300	100,000	$\approx 1.00 \times 10^6$
SBUX	1.00% of value	100	300	100,000	$\approx 1.00 \times 10^6$

4.5 Settlement into an admin-less vault

The pool deposits $R(c)$ of the brand asset into the brand’s ERC-4626 vault [10] with receiver w . With total assets A and share supply S , OpenZeppelin’s virtual-share accounting with decimals offset $\delta_v = 6$ mints

$$\text{shares}(R) = \left\lfloor R \cdot \frac{S + 10^{\delta_v}}{A + 1} \right\rfloor. \quad (10)$$

The offset makes a first-depositor inflation attack unprofitable, which a dedicated vault test checks. The vault has no owner, pause or sweep. Pausing the registry stops new claims but never redemptions.

4.6 Bounded loss under key compromise

Proposition 2 (Attester-compromise bound). *Suppose an adversary obtains the attester key at time t_0 and the owner pauses the registry at $t_0 + \Delta$. Then the total value the adversary can extract from brand b is at most*

$$L_b \leq \min\left(B_b, \left\lceil \Delta/86400 \right\rceil \cdot C_b^{\text{brand}}\right), \quad (11)$$

and shares minted before t_0 are unaffected.

Proof. Every acceptance passes through the registry, which consumes $D_b(\delta)$ and B_b (Eqs. 9, 6). The per-user cap gives no protection, since fresh wallets are free, so the binding limits are the per-brand daily cap over the days touched and the funded budget. No component holds authority over minted shares (Table 1). $\square$

With live parameters, a compromise left unanswered for a full day costs at most 100,000 demo units per brand, and draining a budget takes about ten days. The response requires no redeploy: pause, replace the key in the Stylus verifier’s attester allowlist (`set_attester`), unpause. The same bound covers abuse of the public demo POS.

5 Merchant-Signed Receipts

5.1 Canonical message

A merchant receipt is the field vector

$$\mathbf{f} = (\text{merchantId}, \text{merchantName}, \text{brand}, \text{receiptId}, \text{amount}, \text{currency}, t_{\text{iss}}, t_{\text{exp}}).$$

Each field f_i must match a fixed grammar Γ_i . For example, the amount is a positive integer of at most 14 digits in minor units, and the receipt identifier is 8 to 64 characters from `[A-Za-z0-9-]`. Validity also requires $t_{\text{exp}} > t_{\text{iss}}$ and a validity window of at most 30 days. The signed bytes are

$$M(\mathbf{f}) = \text{"STOCKBACK-MERCHANT-RECEIPT-V1"} \parallel \left\|_{i=1}^8 (\text{"\n"} \parallel k_i \parallel \text{"="} \parallel f_i), \quad (12)$$

and the receipt travels as `SBR1.b64url(f).b64url( $\sigma_M$ )` with $\sigma_M = \text{Sign}_{sk_M}(M(\mathbf{f}))$.

Lemma 1 (No field injection). *No Γ_i admits a newline or `=`, so M is injective on valid field vectors: two distinct valid vectors never produce the same message.*

Proof. Every field is preceded by a newline and a fixed key in a fixed order, and no field value can contain either delimiter. The message therefore parses uniquely back into $\mathbf{f}$. $\square$

By the lemma and the unforgeability of Ed25519 [11], changing any field, including the amount, brand, receipt identifier or either timestamp, invalidates σ_M , which is goal G1.

5.2 Verification and attestation

The attester holds only the merchant public key pk_M . Given a payload it computes

$$\text{ok}(\mathbf{f}, \sigma_M) \iff \text{Ed25519.Verify}_{pk_M}(M(\mathbf{f}), \sigma_M) \wedge \text{issuedAt} \leq t + 120 \wedge t < \text{expiresAt} \wedge \text{brand} \in \mathcal{B}, \quad (13)$$

checking the signature *first* so the more specific errors cannot be used to probe unsigned edits. It then derives every reward-relevant claim field from $\mathbf{f}$, namespacing the receipt reference by merchant ($\rho = \text{merchantId} \parallel : \parallel \text{receiptId}$) so two merchants cannot collide on one nullifier, and clamping the claim deadline to the receipt's expiry, $t_d = \min(t + 3600, \text{expiresAt})$. Before returning the attestation it dry-runs Eq. 6 and refuses a receipt whose nullifier is already burned. Table 4 lists every rejection and the test that exercises it.

Table 4: Merchant receipt rejections. Each is covered by a unit test (20 in total) and, where marked, by the live integration suite against the deployed contracts.

Input	Outcome	HTTP	Live test
Any of 8 fields altered after signing	bad_signature	401	amount, brand, receiptId, issuedAt
Random 64-byte signature	bad_signature	401	yes
Signed by an untrusted key	bad_signature	401	yes
Expired receipt	expired	410	yes
Validly signed, unsupported brand	unsupported_brand	422	yes
Malformed or non-string payload	malformed	400	yes
Receipt already claimed (any wallet)	already_claimed	409	yes, after a real claim
Merchant or attester key missing	fail closed	503	yes

The QR encodes a link whose payload sits in the URL *fragment* (`/app/scan#r=...`), so it is never sent to a server log when the link is opened, and the page removes the fragment after reading it.

6 Ed25519 Verification on Arbitrum Stylus

The EVM has precompiles for keccak and secp256k1 but none for Ed25519, the scheme most payment and service infrastructure signs with. A Solidity verifier must execute SHA-512 and curve25519 arithmetic in bytecode. STOCKBACK's verifier is a Rust contract on Arbitrum Stylus (ed25519-dalek 2.x, verify_strict, stylus-sdk 0.10.9, 20,344 bytes of compressed WASM) behind the same `verify(bytes32,bytes)` interface as the Solidity verifier, so the registry can switch between them with one owner call. Strict verification rejects non-canonical and small-order encodings, so signatures are not malleable.

6.1 Measured gas

Both verifiers were deployed on Robinhood Chain testnet with the same allowlisted key and measured with `eth_estimateGas` on byte-identical calldata over 100 shared test vectors (Table 5). A linear model $G(n) = \alpha + \beta n$ fits both with $R^2 > 0.9999$:

$$G_{\text{sol}}(n) \approx 128,404 + 608,542 n, \quad G_{\text{sty}}(n) \approx 74,139 + 65,088 n. \quad (14)$$

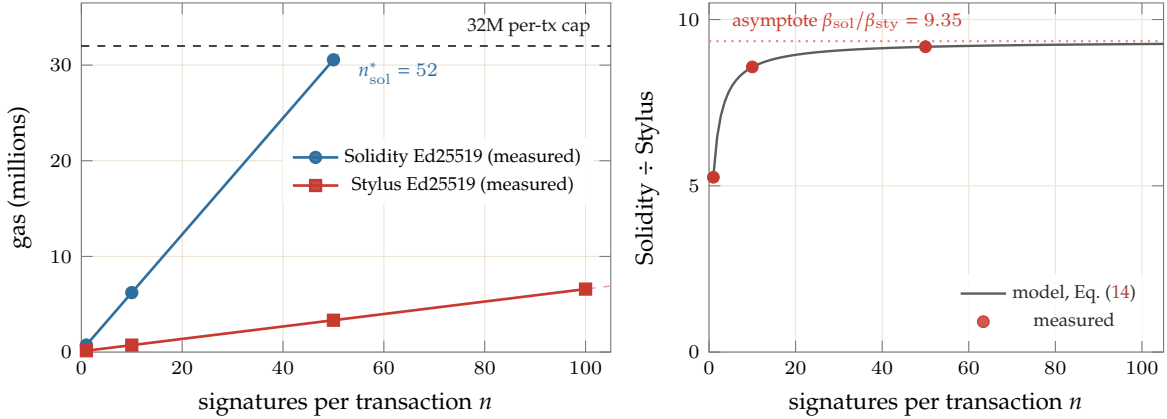
The marginal ratio $\beta_{\text{sol}}/\beta_{\text{sty}} \approx 9.35$ is the asymptotic advantage; the measured ratio approaches it from below as fixed per-transaction costs amortize (Figure 2, right). Under the per-transaction cap $G_{\text{max}} = 32 \times 10^6$,

$$n^* = \left\lfloor \frac{G_{\text{max}} - \alpha}{\beta} \right\rfloor : \quad n_{\text{sol}}^* = 52, \quad n_{\text{sty}}^* = 490 \text{ (model extrapolation)}. \quad (15)$$

For a single claim the difference is smaller but real. A full `submitClaim` cost 278,965 gas with the ECDSA verifier and 372,294 with the Stylus Ed25519 verifier on chain, against 951,760 with the Solidity Ed25519 verifier on a local EVM. ECDSA through the `ecrecover` precompile remains cheapest (about 5,000 gas per signature marginal); Stylus earns its place precisely for schemes the EVM lacks.

Table 5: Gas for verifyBatch on Robinhood Chain testnet (28 September 2026), identical calldata.

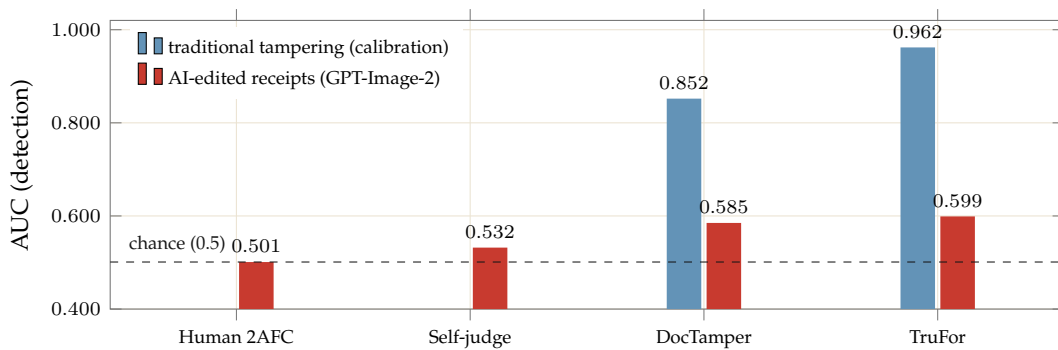
Signatures n	Solidity Ed25519	Stylus Ed25519	Stylus / Solidity	Solidity ÷ Stylus
1	736,502	140,063	0.190	5.3×
10	6,214,368	724,659	0.117	8.6×
50	30,555,405	3,327,511	0.109	9.2×
100	exceeds cap	6,583,441	n/a	n/a

**Figure 2:** Left: measured gas for strict Ed25519 verification with the linear fits of Eq. (14) dashed. Solidity crosses the 32M per-transaction cap at 53 signatures; Stylus uses 21% of it for 100. Right: the cost ratio rises toward the marginal-cost asymptote as fixed costs amortize.

7 Why a Signature, and Why Not Yet a Hold Period

7.1 Pixels cannot be the trust origin

Figure 3 reproduces the detection results of Wu et al. [1]. The two forensic detectors retain near-published performance on traditional splicing of the same documents and lose 0.27–0.36 AUC when the edit is made by a current image model. No judge, human or machine, exceeds 0.60. An attester that reads pixels is therefore a forgery-to-signature converter: every downstream check, however strong, faithfully certifies a fake. This is why tier-2 evidence is labelled as unestablished everywhere it appears, and why the merchant signature is the trust origin of tier 1.

**Figure 3:** Forgery detection collapses on AI-edited receipts. Data from Wu et al. [1] ($n = 3,066$ forgeries; humans $N = 120,365$ pair votes).

7.2 Deterrence needs time

Javarone, Leonardos and Ventre [5] model a farmer who creates n linked identities and submits fraudulent claims for a reward R that vests after T periods, while the issuer audits each claim per period with probability a and detects fraud with probability s . With q_F the per-period eligibility persistence of a fraudulent claim, k_F the cost per identity and F a forfeitable penalty, farming is deterred whenever

$$q_F^T R \leq k_F + T L(a) F, \quad L(a) = -\ln(1 - as), \quad (16)$$

which gives the minimum audit rate for a vesting horizon T :

$$a_{\min}(T) = \frac{1}{s} \left[1 - \exp\left(-\frac{q_F^T R - k_F}{T F}\right) \right]. \quad (17)$$

At $T = 0$ no audit rate deters: a claim that pays out immediately has no window in which to be checked. STOCKBACK settles in the claim transaction, so $T = 0$ today. Its defence is instead to raise k_F , the cost of a fraudulent claim, by requiring a merchant signature, and to cap R (Eq. 8). Figure 4 plots Eq. (17) with the authors' own calibration to show what a hold period would buy. It motivates the planned hold-and-void escrow in Section 12; it is not a measurement of STOCKBACK.

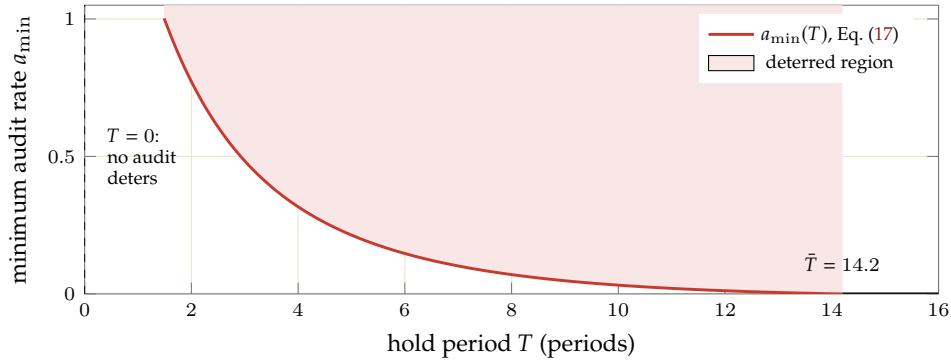


Figure 4: Deterrence frontier of Eq. (17) under the calibration of [5] ($R = 100$, $k_F = 10$, $q_F = 0.85$, $s = 0.6$, $F = 50$). Longer holds need less auditing; with no hold, nothing deters.

7.3 Replay is solved; Sybil resistance is not

Proposition 1 prevents a receipt from counting twice. It does not tie a wallet to a person: Luo et al. found that 69.3% of reported airdrop-hunter groups on Hop would have profited had they not been disqualified [6]. STOCKBACK's answer is structural rather than behavioural. Rewards attach to receipts, not identities, so a "Sybil" holding many *genuinely merchant-signed* receipts is simply a frequent customer. That argument holds only while the merchant key is not compromised and, in this deployment, does not hold at all because the demo POS is public (Section 10).

8 Implementation and Evaluation

Code. 791 lines of Solidity across eight contracts and their interfaces, 1,263 lines of Solidity tests, 251 lines of Rust, and about 4,600 lines of TypeScript for the web application, attester and simulated POS.

Tests. Table 6 lists every suite and what it runs against. All results are from runs on 2 October 2026.

On-chain activity. As of 2 October 2026 the registry has settled **12 claims** from **3 wallets** (10 Nike, 1 Apple, 1 Starbucks), allocating 229.45 demo units in total. Representative transactions:

Table 6: Test suites. “Live” suites run against the deployed contracts on Robinhood Chain testnet.

Suite	Count	Covers	Result
Foundry (Solidity)	83	Unit, integration, security; 3 fuzz tests at 1,000 runs; 5 invariants at 128×64 calls (no replay, conservation, solvency, caps)	all pass
Rust (Stylus)	5	Strict Ed25519 on all 100 shared vectors; tampered digest, signature, length	all pass
Receipt signing (Node)	20	Valid receipt; tampering of each of 8 fields; wrong key; expiry; injection; fail-closed	all pass
Live integration	22	API rejections, previewClaim, a real submitClaim, duplicate rejection, keys missing	all pass
Browser (Chromium)	11	POS → QR link → verify → claim → replay rejected, real wallet	all pass
Secret scan	1	No secret value in 679 client build files or the git-tracked tree (negative control verified)	pass

Claim	Transaction	Gas used
Ed25519 attestation via Stylus (Apple, ₹14,990)	0x7fe2a51a...d9c4	372,294
Merchant-signed receipt, browser (Nike, ₹2,000)	0xabd3b62b...5388	290,374
Merchant-signed receipt, API suite (Nike, ₹2,000)	0x801cac69...5627	288,025

The first Stylus claim cost more than later ones; we attribute the difference to first-write storage on that brand’s cap counters and vault, but we did not isolate it and report it as an observation, not a result.

9 Findings from Building Against Deployed Code

Each finding below would have shipped as a silent defect had we trusted documentation or local tests alone.

- The upstream swap interface did not match the deployed router.** The forked code’s `ISwapRouter` used the `Uniswap V3` struct with a `deadline` field (selector `0x414bf389`). The router configured for Robinhood Chain mainnet does not contain that selector in its bytecode; it contains the `SwapRouter02` selector `0x04e45aaf`. Every swap through it would have reverted. STOCKBACK’s USDG adapter uses the `SwapRouter02` layout, verified against bytecode.
- A naive Solidity Ed25519 loop is quadratic.** Without rewinding the free-memory pointer between signatures, the library’s allocations accumulate: 814M gas for 100 signatures against 60M with the rewind. Only the latter is a fair baseline, and it is the one we report.
- Clipping a capped reward burns the receipt.** An early draft clipped rewards at the daily caps. Because the nullifier is burned on acceptance, a clipped claim would destroy the rest of the receipt’s value; caps now reject (Eq. 9).
- Receipt identifiers collide across merchants.** Two merchants can issue the same invoice number. The attester namespaces the reference as `merchantId:receiptId` before salting, so each merchant has its own nullifier space.
- An attestation must not outlive its receipt.** A one-hour claim deadline on a receipt that expires in ten minutes would extend the receipt’s life on chain. The deadline is clamped to the receipt’s expiry.
- A second receipt link only changes the fragment.** Opening a new receipt link while already on the scan page changes only `location.hash`, so the page did not react. The browser suite caught it; the page now listens for `hashchange`.
- In-memory rate limits are per instance.** Consecutive integration runs exhausted a per-IP limit of 20 requests per 10 minutes and failed with 429s. The behaviour is correct, but on a serverless host each instance counts separately; it is listed as a limitation.

10 Trust Assumptions and Limitations

Stated plainly, in decreasing order of weight.

1. **The merchant is simulated, and its POS is public.** Anyone can mint a validly signed demo receipt. Tier 1 here demonstrates the mechanism, not fraud resistance; in production the key lives in the merchant's POS or HSM.
2. **No hold period.** Rewards settle immediately, so refunded purchases keep their reward and no audit can claw anything back ($T = 0$ in Eq. 16).
3. **Single attester key, single owner key.** Compromise is bounded by Proposition 2, not prevented. There is no k-of-n attestation, timelock or multisig yet.
4. **A receipt QR is a bearer token.** Whoever submits it first claims it. Copying an *attestation* is useless, since the claimant is signed, but copying the *receipt* before the customer claims is not prevented.
5. **No proof of personhood.** Replay is prevented per receipt; Sybil resistance rests on the merchant key.
6. **Mock assets.** Every brand asset and USDG on testnet is a labelled mock that refuses to deploy on production chains. Shares are not securities, and no brand partnership exists or is implied.
7. **Amounts are public.** Merchant hash, amount and time are visible on chain; only identifiers are hidden.
8. **Unaudited code.** The Solidity contracts and the Stylus verifier have not been externally audited.

11 Related Work

Document forensics. GPT4o-Receipt [2] found that LLM judges detect fully generated receipts mainly through arithmetic errors; AIForge-Doc v2 [1] shows that localized AI edits defeat humans and detectors alike; FraudBench [3] extends this to refund evidence. STOCKBACK does not try to detect forgeries; it removes the image from the trust path. **Provable web data.** DECO [8] proves that data came from a TLS server without server cooperation, in about 5 seconds online and 3–13 seconds of proving, and is the natural basis for tier 3. **Reward mechanism design.** Javarone et al. [5] characterize deterrence under vesting and stochastic audit; Luo et al. [6] measure hunter profits in airdrops; Yin and Wen [4] model trust arbitrage under generative AI. **Loyalty architecture.** Oamen et al. [7] attribute coalition failure to the central operator and propose brand-sovereign programs settled through trustless contracts and a universal asset; STOCKBACK's per-brand vaults and USDG funding follow that architecture. **Primitives.** EIP-712 [9], ERC-4626 [10], RFC 8032 [11] and Arbitrum Stylus [12].

12 Future Work

A hold-and-void escrow that releases shares after a per-brand return window and lets a merchant-signed refund void the claim, making Eq. (16) satisfiable; a merchant key registry with rotation and revocation; k-of-n attestation verified as a batch in Stylus, where Eq. (15) leaves ample room; tier-3 evidence from zkTLS proofs of order and payment records; amount privacy through range proofs; and, before any mainnet deployment, an external audit, a timelocked multisig owner and issuer-authorized assets behind the existing jurisdiction adapter.

13 Conclusion

A reward is only as honest as the evidence it pays on, and a photograph is no longer evidence. STOCKBACK makes the trust origin a signature, labels every claim with what was actually established, settles in one transaction whose checks are verified in Rust on Arbitrum at up to 9.2× less gas, and pays into vaults no one can administer. The design bounds what any single key can lose and states what it does not yet do. Every number in this paper was measured on the deployed system or taken from the cited work.

References

- [1] J. Wu, Y. Zhou, D. T. Ng, X. Shen, K. Zewde, A. Raj, T. Duong, S. Ren. *When the Forger Is the Judge: GPT-Image-2 Cannot Recognize Its Own Faked Documents*. arXiv:2604.25213, 2026.
- [2] Y. Zhang, S. Ren, et al. *GPT4o-Receipt: A Dataset and Human Study for AI-Generated Document Forensics*. arXiv:2603.11442, 2026.
- [3] X. Yan, B. Chen, et al. *FraudBench: A Multimodal Benchmark for Detecting AI-Generated Fraudulent Refund Evidence*. arXiv:2605.08820, 2026.
- [4] X. Yin, F. Wen. *When Trust Attracts Fraud: AI and Trust Arbitrage*. Economics Letters; arXiv:2609.27404, 2026.
- [5] M. A. Javarone, S. Leonardos, C. Ventre. *NFT-Based Reward Mechanisms: Sybil Farming, Vesting, and Stochastic Verification*. arXiv:2609.13064, 2026.
- [6] J. Luo, H. Kang, S. Zheng, X. Liu. *Toward Resilient Airdrop Mechanisms: Empirical Measurement of Hunter Profits and Airdrop Game Theory Modeling*. arXiv:2503.14316, 2025.
- [7] P. O. Oamen, R. Wesley, P. Onobhayedo. *Orchestrating Rewards in the Era of Intelligence-Driven Commerce*. arXiv:2512.00738, 2025 (citing the Bond Brand Loyalty Report 2020).
- [8] F. Zhang, D. Maram, H. Malvai, S. Goldfeder, A. Juels. *DECO: Liberating Web Data Using Decentralized Oracles for TLS*. ACM CCS 2020; arXiv:1909.00938.
- [9] R. Bloemen, L. Logvinov, J. Evans. *EIP-712: Typed Structured Data Hashing and Signing*. Ethereum Improvement Proposals, 2017.
- [10] J. Feng et al. *ERC-4626: Tokenized Vaults*. Ethereum Improvement Proposals, 2021.
- [11] S. Josefsson, I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, IETF, 2017.
- [12] Offchain Labs. *Arbitrum Stylus Documentation*. docs.arbitrum.io, 2026.
- [13] Project STOCKBACK. *Source, tests, benchmarks and deployment records*. github.com/notwen123/STOCKBACK, 2026.

STOCKBACK is hackathon software on a public testnet. Gas figures were measured on Robinhood Chain testnet (chain 46630) on 28 September 2026; reward parameters, claim counts and test results on 2 October 2026. Contract addresses are in `deployments/46630.json`: registry `0x4273b12c04A65c2180d4e65Bcb4254825cE64120`, Stylus verifier `0x9ae8a390121ba71545e9923b333d60e7e3ccd3bd`. The merchant in the demo is simulated, all brand assets are mocks, and no brand partnership exists or is implied. Figure 3 and Figure 4 use data and parameters from the cited papers, not STOCKBACK measurements. Nothing in this paper claims a result that was not measured.